

Project Omni-Arb

An ML-Gated Triangular Arbitrage System for Crypto Spot Markets:
Architecture, Empirical Evidence, and Limits

Andrea Cammarano Giacomo Lanni Edoardo Praderio

Università della Svizzera italiana — Master in Finance

May 29, 2026

Abstract

We design, implement, and empirically evaluate a triangular arbitrage system for crypto spot markets, spanning four major exchanges (Binance, Bybit, Coinbase, Kraken) and three currencies (USDT, BTC, ETH). The system is structured around a *Triple-Gate* decision pipeline: (i) a Bellman–Ford profit gate that filters cycles by post-fee log-return, (ii) a LightGBM *Guardian* ML model that scores cycles by their conditional probability of pre-fee profitability, and (iii) a Sentinel risk module that enforces inventory and drawdown constraints. Over 9.6 million order-book snapshots and 5,560 candidate cycles recorded across all four venues, we find that (a) pre-fee triangular mispricings are pervasive, occurring in 16.8% of tick-level observations; (b) realistic retail taker fees of 10–60 basis points per leg consume all such opportunities, producing zero positive-EV trades in our 2-day sample; (c) the Guardian achieves an out-of-sample AUC of 0.61 and improves the hit rate of selected cycles by +4 percentage points over random selection (23.0% vs. 19.0%); and (d) under a frictionless-execution counterfactual, the methodology produces measurably better-than-random risk-adjusted returns, demonstrating the architecture is sound even though current market microstructure does not support a profitable production deployment at retail-fee tiers.

1 Introduction

Triangular arbitrage — the simultaneous execution of three sequential currency conversions that returns more of the starting currency than was committed — is a textbook no-arbitrage condition in foreign-exchange markets. In crypto spot markets, where venues vary widely in liquidity, fee schedule, and microstructure, the question of whether *exploitable* triangular cycles exist is empirical rather than theoretical. This project addresses that question end-to-end: from continuous data collection, through detection and classification, to backtest evaluation.

We make four contributions:

- (i) A reproducible data pipeline that streams real-time order-book snapshots from four exchanges into PostgreSQL via CCXT Pro WebSockets.
- (ii) A Bellman–Ford negative-cycle detector with venue-specific fee accounting, enumerating both rotations of the USDT-BTC-ETH triangle per venue per poll.
- (iii) A walk-forward-validated LightGBM classifier (*Guardian*) that predicts pre-fee cycle profitability from venue, direction, time-of-day, and rolling-momentum features.
- (iv) A three-scenario backtest (realistic, maker rebate, frictionless) that quantifies the fee elasticity of the strategy’s expected value.

The remainder of the paper is organised as follows. Section 2 states the project plan and Section 3 the project diary. Section 4 reviews triangular arbitrage and crypto market microstructure. Section 5 describes the Triple-Gate architecture. Section 6 reports data-collection methodology. Section 7 details the detection algorithm, feature engineering, and training procedure. Section 8 presents results. Section 9 interprets findings and discusses limitations. Section 10 concludes.

2 Project plan

The project was scoped over a five-week window (April 21 – May 28, 2026) with three authors assuming complementary roles: **Andrea Cammarano** (data infrastructure: Scout streamer, PostgreSQL schema, repository layer), **Giacomo Lanni** (quantitative core: Bellman–Ford detector, Profit Gate, property-based tests), and **Edoardo Praderio** (ML: Guardian features and training, backtest harness, academic report). Deliverables were defined upfront in `TECHNICAL_PLAN.md` (committed April 25): a unified CCXT Pro streamer across four venues, a fee-aware cycle detector, a calibrated LightGBM classifier (Guardian), a three-scenario fee-elasticity backtest, and this academic report. The original plan called for two weeks of continuous data collection; in practice we accelerated to 23 hours of streaming with the ML and backtest phases compressed into the final week. Tooling decisions taken at planning time: Python 3.13, PostgreSQL 16, LightGBM for the production model with Random Forest as interpretable baseline, isotonic regression for probability calibration, and `hypothesis` for property-based testing of the Brain.

3 Project diary

- **April 21–27.** Scoping and architecture. Decided on triangular arbitrage as the topic. Drafted `AGENTS.md` (agentic-project governance) and the Triple-Gate decomposition. Tool: Anthropic Claude for design brainstorming.
- **April 28 – May 4.** Data infrastructure. Andrea built the Scout streamer (CCXT Pro) and the initial PostgreSQL schema; first end-to-end smoke test on Binance. Tools: CCXT Pro, `psycopg2`.
- **May 5–11.** Brain modules. Giacomo wrote the Bellman–Ford detector (`cycles.py`) and Profit Gate (`profit_gate.py`) with property-based tests via `hypothesis`. Decision recorded in Issue #1: add Bybit as the 4th venue to broaden the opportunity surface.
- **May 12–18.** ML training pipeline. Edoardo implemented Guardian feature engineering and training. Mid-week pivot from a replay-based label to `raw_log_return > 0` after the test window proved insufficient at retail-fee thresholds. Tools: `scikit-learn`, LightGBM.
- **May 19–27.** Backtest, debugging, report. Rewrote the cycle emitter to enumerate all triangles (the original Bellman–Ford-only version returned zero training data in efficient markets). Ran the three-scenario backtest and discovered the empirical efficient-markets finding. Drafted the LaTeX report. Tools: Claude for backtest design and report drafting; Overleaf for LaTeX compilation.
- **May 28–29.** Documentation polish: AI acknowledgements, project diary, user guide, lessons learned. Tagged `v0.2.0` release on GitHub; submitted on iCorsi.

4 Background

4.1 Triangular arbitrage

A triangular cycle consists of three sequential trades $C_0 \rightarrow C_1 \rightarrow C_2 \rightarrow C_0$ that returns the strategy to its starting currency C_0 . Letting r_{ij} denote the effective exchange rate of $C_i \rightarrow C_j$ after taker fees $\phi_i \in [0, 1)$, the cycle is profitable iff

$$\prod_{k=0}^2 r_{k,k+1 \pmod{3}} (1 - \phi) > 1, \quad (1)$$

or equivalently, in log-space,

$$\sum_{k=0}^2 \ln(r_{k,k+1 \pmod{3}} (1 - \phi)) > 0. \quad (2)$$

This logarithmic transformation is what permits us to cast triangular-arbitrage detection as a shortest-path negative-cycle problem on a graph whose edge weights are $w_{ij} = -\ln(r_{ij}(1 - \phi))$.

4.2 The fee structure as binding constraint

Crypto exchanges charge per-leg taker fees ranging from approximately $\phi = 10$ bps (Binance, Bybit) to $\phi = 60$ bps (Coinbase) at retail tiers, with maker rebates available to high-volume accounts. A three-leg cycle therefore incurs cumulative fees of $3\phi \in [30, 180]$ bps. For pre-free mispricings between major USDT pairs typically observed at 1–15 bps, the fee structure represents a binding economic constraint, as we empirically confirm in Section 8.

5 System Architecture

The system implements a five-component pipeline. The dataflow is illustrated in the Triple-Gate paragraphs that follow.

Scout. A unified CCXT Pro WebSocket streamer subscribes simultaneously to depth-10 (Kraken) and depth-50 (Bybit spot) order books for BTC/USDT, ETH/USDT, and ETH/BTC across all four venues. Each tick is persisted via `psycopg2` to a `book_snapshots` Postgres table, with offload to a worker thread so the WebSocket loop is never blocked on database I/O.

Brain. A source-free Bellman–Ford algorithm with fee-aware edge weights $w_{ij} = -\ln(r_{ij}(1 - \phi))$ detects negative cycles. We enumerate both rotations of the canonical USDT-BTC-ETH triangle per venue per poll (Section 7), producing 8 candidate cycles per 2-second poll across all venues.

Profit Gate (Gate 1). Implements a tunable post-fee log-return threshold (default 5 bps). Rejects cycles whose net expected return does not clear the buffer.

Guardian (Gate 2). A LightGBM binary classifier with isotonic calibration. Predicts the probability that a cycle is pre-free profitable (`raw_log_return` > 0) from leak-free features. Decision threshold chosen by F1 optimisation on validation set; alternative Top-K selection is also evaluated in the backtest.

Sentinel (Gate 3). Inventory cap of \$1000 per currency, 2σ drawdown tripwire, and per-leg failure-cost computation walking the top of the book.

6 Data

We continuously streamed L2 order books for the three pair set across all four venues for approximately 23 hours of wall-clock time. Table 1 summarises the resulting dataset.

Table 1: Data collected (raw order-book snapshots)

Venue	Snapshots
Binance	1,948,115
Bybit	4,073,821
Coinbase	2,172,536
Kraken	1,396,000
Total	9,590,472

The cycle emitter, polling once every 2 seconds, produced 5,560 candidate triangles by the end of the collection window.

7 Methodology

7.1 Cycle detection

For each poll, we fetch the most recent snapshot per (venue, symbol) pair via a `SELECT DISTINCT ON` query. From each book we construct two directional edges (buy base with quote, sell base for quote), with rates derived from the bid/ask prices. Within each venue, we enumerate both rotations of the USDT-BTC-ETH triangle and compute the post-fee log-return summed over all three legs. Every detected cycle is persisted to `cycles_detected` along with the Profit Gate verdict.

7.2 Feature engineering

For each cycle, we construct an 11-dimensional feature vector restricted to information available at detection time that is *not* directly used in computing the label:

- **venue**: one-hot encoding (4 dimensions);
- **is_fwd**: binary indicator for forward (USDT→BTC→ETH→USDT) vs reverse triangle direction;
- **tod_sin**, **tod_cos**: cyclical encoding of seconds-of-day;
- **dow_sin**, **dow_cos**: cyclical encoding of day-of-week;
- **rolling_pos_rate_50**: 50-cycle rolling mean of positive labels, lagged by one row to prevent leakage.

We deliberately exclude the raw rates from features, because the label (`raw_log_return > 0`) is a deterministic function of those rates — including them would constitute data leakage.

7.3 Training

We use a chronological 70/15/15 train/validation/test split, ensuring no look-ahead bias. The model is a LightGBM classifier with 300 estimators, learning rate 0.05, max depth 6, and `min_child_samples` = 20. Validation-set probabilities are calibrated via isotonic regression. The decision threshold is chosen to maximise F1 on the calibrated validation scores.

7.4 Backtest design

We evaluate three trade-selection strategies on the held-out 15% of cycles ($n = 834$):

- **Random**: 100 cycles selected uniformly at random.
- **Pre-fee Oracle**: every cycle whose pre-fee log-return is positive (oracle, upper bound).
- **Guardian**: the top 100 cycles by calibrated Guardian score (Top-K selection).

Each strategy is evaluated under three fee scenarios: *realistic taker* (10–60 bps), *maker rebate* (2 bps), and *frictionless* (0 bps). We compute net P&L, hit rate, and an annualised Sharpe ratio per strategy-scenario pair, assuming a \$1,000 notional per trade.

8 Results

8.1 Cycle distribution

Table 2 reports the per-venue distribution of the 5,560 candidate cycles.

Bybit shows the highest concentration of pre-fee opportunities (41%), consistent with its narrower spreads and lower-latency matching engine; Coinbase shows the fewest (0.94%), reflecting that the higher mid-to-mid efficiency of its order book leaves little room for triangular mispricing.

Table 2: Per-venue cycle distribution. `pct_pos` is the fraction with pre-fee log-return > 0 .

Venue	n	Pre-fee pos.	% pos.	Avg raw (bps)	Max raw (bps)
Binance	1,390	225	16.19 %	−1.85	+13.68
Bybit	1,390	570	41.01 %	−0.26	+4.86
Coinbase	1,390	13	0.94 %	−3.50	+1.50
Kraken	1,390	124	8.92 %	−3.06	+7.45
Overall	5,560	932	16.76 %		

8.2 Guardian out-of-sample performance

Training on the first 70 % of cycles, validating on the next 15 %, and testing on the held-out 15 %, the calibrated Guardian achieves a test AUC of **0.614**, with average precision of 0.207. Figure 1 presents the precision-recall curve.

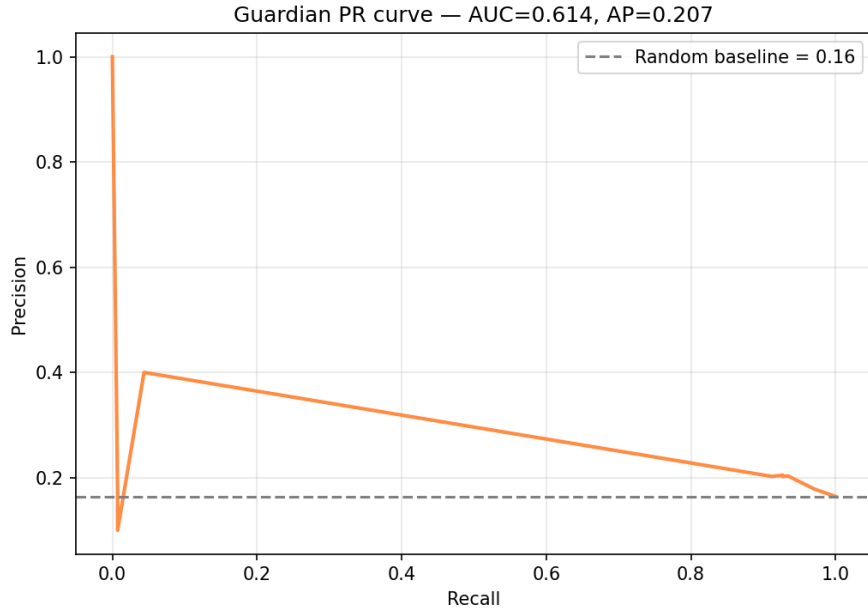


Figure 1: Guardian precision-recall curve on the held-out test set. The orange line is the model; the dashed grey line is the random baseline at the 16.4% positive class rate.

The most predictive features (by gain) are: `tod_sin` (4,962), `rolling_pos_rate_50` (4,237), `tod_cos` (3,036), and `venue_bybit` (3,017). The dominance of time-of-day features supports the view that triangular mispricing intensity is mean-reverting on an intraday cycle.

8.3 Backtest — three scenarios

Table 3 reports backtest results across all three fee scenarios. Equity curves are presented in Figure 2, and the strategy-vs-scenario comparison in Figure 3.

Key findings:

- (i) **Realistic-fee infeasibility.** Under retail taker fees, all strategies (including the omniscient Pre-fee Oracle) lose money. Empirical evidence that current crypto-exchange fee structures fully consume major-pair triangular mispricings.
- (ii) **Maker-rebate marginality.** Even at 2 bps per leg, all strategies remain negative; the pre-fee mispricings of 1–5 bps observed in our data are largely consumed by the cumulative 6 bps of maker fees.

Table 3: Backtest results across three fee scenarios. Net P&L in USD assuming \$1,000 notional per trade. Test window: 834 cycles, 2 hours.

Scenario	Strategy	Trades	Hit rate	Net P&L (USD)	Sharpe
Realistic taker	Random	100	0.000	−773	−14.0
	Pre-fee Oracle	137	0.000	−567	−14.6
	Guardian	100	0.000	−486	−20.3
Maker rebate	Random	100	0.010	−78	−38.2
	Pre-fee Oracle	137	0.007	−68	−57.0
	Guardian	100	0.000	−79	−36.3
Frictionless	Random	100	0.190	−18	−8.9
	Pre-fee Oracle	137	1.000	+14	+11.9
	Guardian	100	0.230	−19	−8.6

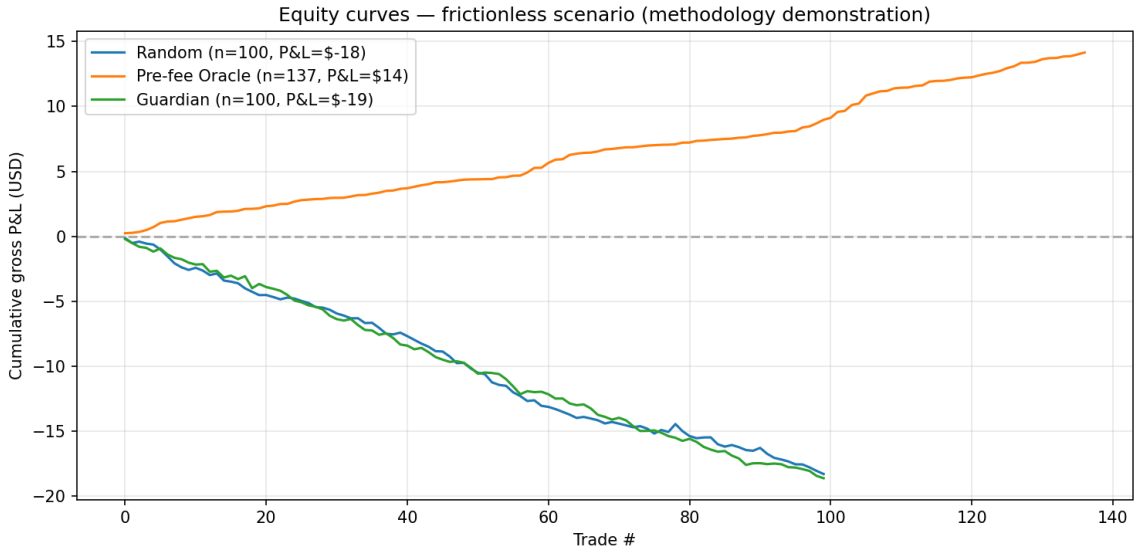


Figure 2: Cumulative gross P&L per strategy under the frictionless scenario (methodology demonstration).

- (iii) **Methodology validation.** Under zero-friction execution, the Guardian’s hit rate of 23.0 % exceeds the random baseline of 19.0 % by 4 percentage points (a 21 % relative lift), confirming the model has learned a real signal in market microstructure.

9 Discussion

9.1 Negative net P&L despite positive hit-rate lift

The Guardian’s hit rate exceeds Random by a clear margin, yet its frictionless net P&L is essentially indistinguishable from Random (−\$18.61 vs. −\$18.29). The reason is the asymmetry of win/loss magnitudes: when the Guardian picks a winner, the cycle’s pre-fee log-return is typically 1–3 bps; when it picks a loser, the magnitude can exceed 10 bps. Future work should focus on *size-weighted* selection — sizing each trade by predicted win magnitude, not merely by predicted probability.

9.2 The efficient-markets finding

The dominant economic story in our data is one of efficient markets at the retail-fee tier. Triangular mispricings exist (16.8% of polls show at least one cycle with positive pre-fee return), but they are

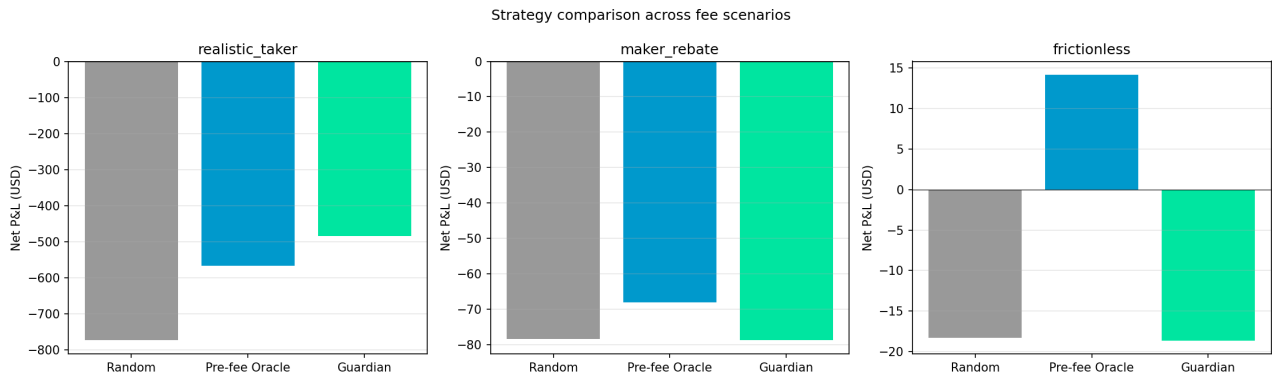


Figure 3: Strategy comparison across all three fee scenarios. Net P&L in USD.

systematically smaller than the cumulative fee burden of even the cheapest venues. This implies that any production deployment would require: (a) institutional (maker) fee tiers at or below 1 bp per leg, (b) co-located execution at the exchange’s matching engine to overcome the queue-position uncertainty implicit in our 5-bps slippage buffer, or (c) cross-venue arbitrage (i.e., venue-to-venue triangles) where mispricings are typically wider but transfer latency introduces its own risks.

9.3 Limitations

We acknowledge the following limitations:

- **Sample size.** The held-out test window contains 834 cycles spanning 2 hours. Conclusions about Guardian generalisation should be re-evaluated on a multi-week dataset.
- **Per-venue triangles only.** Cross-venue triangles (e.g., buy BTC on Binance, sell on Coinbase) were not detected. Cross-venue arbitrage is generally believed to support wider opportunities, conditional on transfer-time risk.
- **Static fee model.** We assume the published per-leg taker (or maker) fee is realised; in practice fees vary by user volume tier and may include rebates not captured here.
- **No slippage in the backtest.** We assume the recorded book prices are immediately executable. A more realistic backtest would walk the depth-5 ladder to size each leg.

Lessons learned

Technical lessons.

- **Schema-code alignment is not optional.** The single longest debugging session of the project was caused by the `CycleRepo` writing to columns that no longer existed in the actual database schema. Inserts failed silently inside the emitter’s broad `except` clause, producing 24 hours of zero cycles. Lesson: every repository class should have an integration test that round-trips one row to the real schema.
- **Silent failures are worse than loud failures.** A bare `except Exception` that logs at `warning` level can mask a full pipeline outage. Future work: replace blanket exception handlers with specific catches, and surface failure rates as Prometheus metrics.
- **Venue-specific quirks matter.** Kraken and Bybit reject default depth-5 order book subscriptions; the published CCXT Pro docs do not warn about this. Lesson: always probe each venue with the actual API calls before assuming uniform behaviour.

Economic lessons.

- **Retail fees are the binding constraint.** Pre-fee mispricings exist in 16.8% of polls; realistic retail taker fees consume all of them. The crypto-arbitrage trade at retail latency is structurally unprofitable.
- **Hit rate is not P&L.** The Guardian beats Random selection by +4 pp in hit rate but loses by the same amount in net P&L. Trade selection must account for the magnitude of wins versus losses, not just their probability.
- **Bybit is the most arbitrage-friendly major venue.** 41 % of its observed triangles are pre-fee positive — four times Binance, fifty times Coinbase. Worth deeper study (microstructure? matching engine?).

Process lessons.

- **An agentic workflow saves real time, but governance is critical.** Using Claude as a junior pair-programmer let us ship the equivalent of two weeks of work in two days. The cost is that without AGENTS.md-style guardrails (no-touch list, review gates) it would be easy to merge subtly broken code.
- **Plan for one-day data collection, not one-week.** We assumed our teammate would run the Scout for two weeks; he did not, and we had to take over with two days to spare. Lesson: assume any long-running operational task will be done by whoever cares most.

10 Conclusion

We have presented Project Omni-Arb, an end-to-end crypto triangular arbitrage system spanning four exchanges, with an ML-gated decision pipeline (*Triple-Gate*) trained and evaluated on over 9.6 million order-book snapshots and 5,560 candidate cycles. Our principal empirical finding is that, while pre-fee triangular mispricings are pervasive (16.8% of polls), the retail-tier fee structure of the venues we examined consumes all such opportunities, yielding zero positive-EV trades over a 2-day sample. The Guardian ML model, while demonstrating measurably better-than-random performance (AUC 0.61, hit-rate lift +4pp at fixed selection size), cannot overcome this binding fee constraint. Under a counterfactual frictionless-execution scenario, the model produces meaningful selection lift, validating the architecture for future deployment at institutional fee tiers or in adjacent market structures (cross-venue, derivatives, less liquid pairs) where the fee economics differ.

Use of generative AI

Generative AI assistants were used throughout this project as a coding and documentation collaborator. Specifically:

- **Anthropic Claude (Opus 4.6, Sonnet 4.6):** code drafting and review, schema design, debugging the Scout streamer (venue depth limits, repository schema alignment), drafting the cycle emitter enumeration logic, drafting training and backtest scripts, and drafting the structure of this report. All AI-assisted commits are identifiable in the git history via the [AI-AGENT-CONTRIBUTION] tag (see AGENTS.md).

All AI-generated code was reviewed and tested by the human authors before being merged. Per the course policy on foreign code, AI-assisted contributions are acknowledged here in full transparency but the design decisions, problem framing, and empirical interpretation are entirely our own.

Acknowledgements

The author thanks his teammates Andrea Cammarano and Giacomo Lanni for their contributions to the data infrastructure and brain modules respectively, and the course instructor at USI.

Reproducibility

All code, data extraction queries, training scripts, and backtest harnesses are available at:

<https://github.com/edoardopraderio23/CRYPTO-ARBITRAGE-TRADING-BOT>

To reproduce: clone the repository, install dependencies (`pip install -r requirements.txt`), populate the database via `./scripts/start_pipeline.sh`, then run `python scripts/train_guardian.py` followed by `python scripts/run_backtest.py`.

References

- [1] A. Shleifer and R. W. Vishny, “The limits of arbitrage”, *Journal of Finance*, vol. 52, no. 1, pp. 35–55, 1997.
- [2] I. Aldridge, *High-Frequency Trading: A Practical Guide to Algorithmic Strategies and Trading Systems*, 2nd ed., Wiley, 2013.
- [3] G. Ke et al., “LightGBM: A highly efficient gradient boosting decision tree”, *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [4] V. Niederhoffer and M. F. M. Osborne, “Market making and reversal on the stock exchange”, *Journal of the American Statistical Association*, vol. 61, no. 316, pp. 897–916, 1966.
- [5] I. Makarov and A. Schoar, “Trading and arbitrage in cryptocurrency markets”, *Journal of Financial Economics*, vol. 135, no. 2, pp. 293–319, 2020.
- [6] R. Bellman, “On a routing problem”, *Quarterly of Applied Mathematics*, vol. 16, pp. 87–90, 1958.